ACME Delivery Streaming analytics

Architecture, deliverables and measured performance

Fabricio Dutra

From source changes to queryable order metrics.

Sources: README.md | docs/assessment-deliverables.md

DELIVERABLES

What was delivered

Four business queries

Open orders by date and status
Top delivery dates
Pending items by product
Top customers

A complete data pipeline

PostgreSQL → Debezium → Kafka → Spark
Raw events and Delta query tables
Queries available through a CLI

Architecture and sizing

Component diagram and data model
Spark resource budgets
Assumptions and scaling decisions

Code and execution evidence

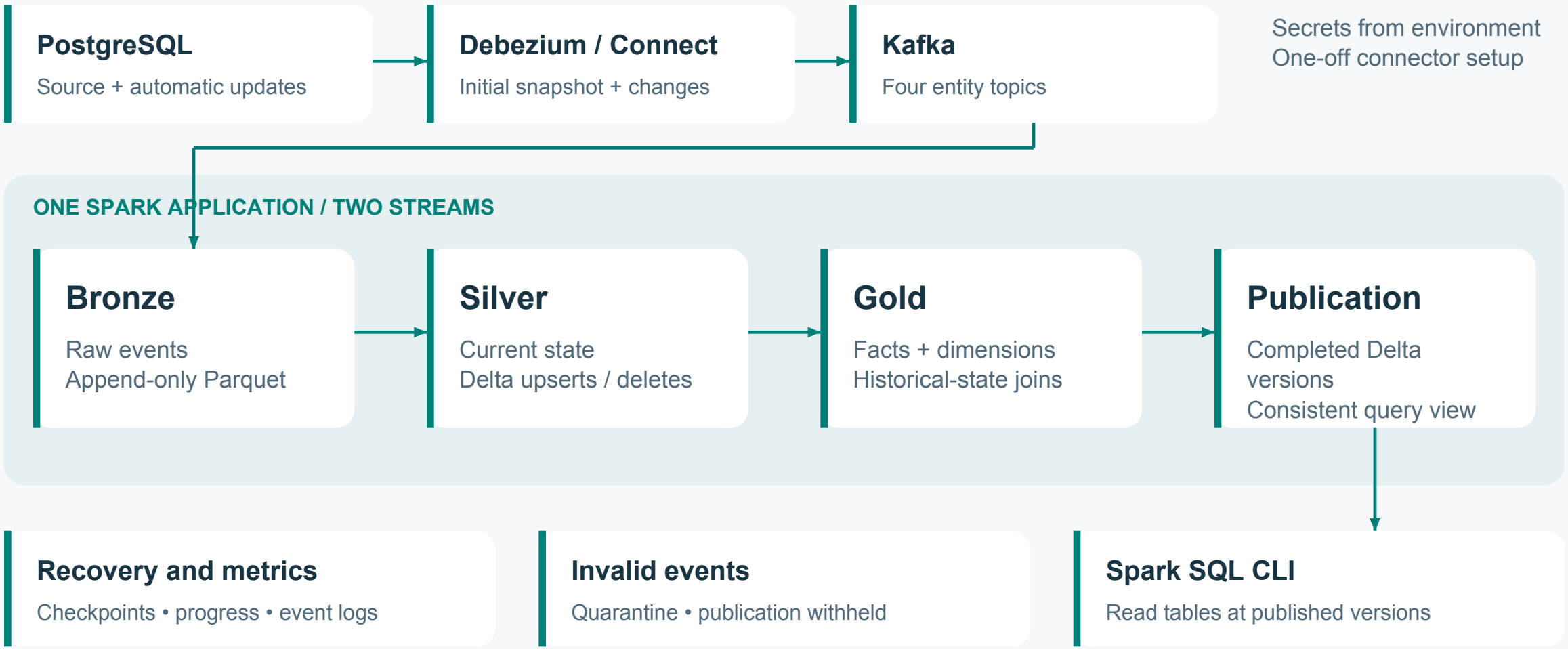
Docker setup, application and SQL
Tests and source/Gold comparisons
Performance results and experiment code

The README links each requirement to its implementation and evidence.

Sources: docs/assessment-deliverables.md | sql/pending-items.sql

ARCHITECTURE

How the components connect



Sources: docs/architecture.md | docker-compose.yaml

How data becomes useful

1 · Keep the event

Bronze: original Kafka bytes

Silver: latest state per key

Ordered updates and deletes

2 · Use stored history

New item → stored order

Changed order → stored items

Earlier batches stay available

3 · Build query tables

One fact row per order
One fact row per item

Customer, product and date dimensions

Consistent reads

Readers see a completed publication. History = snapshot + stored current state.

Sources: app/gold.py | app/materialize.py

SPARK SIZING

The local Spark size and its limits

2 slots
Shared by both streams
local[2] • two-CPU quota

2 GiB
Spark heap
3 GiB container limit

4
Shuffle partitions
Initial setting, not a proven optimum

Setting	Local choice	Why it matters
Streaming trigger	Every 10 seconds	Update frequency vs. repeated job cost
Kafka intake cap	20,000 events per trigger	Roughly 2,000 events/s at this cadence
Scale decisions	Measure CPU, spill and lag	Add capacity or reduce the expensive work

Local sizing was exercised. A production cluster size has not been validated.

Sources: docs/performance.md | docker-compose.yaml

MEASUREMENT

Which performance metrics were measured

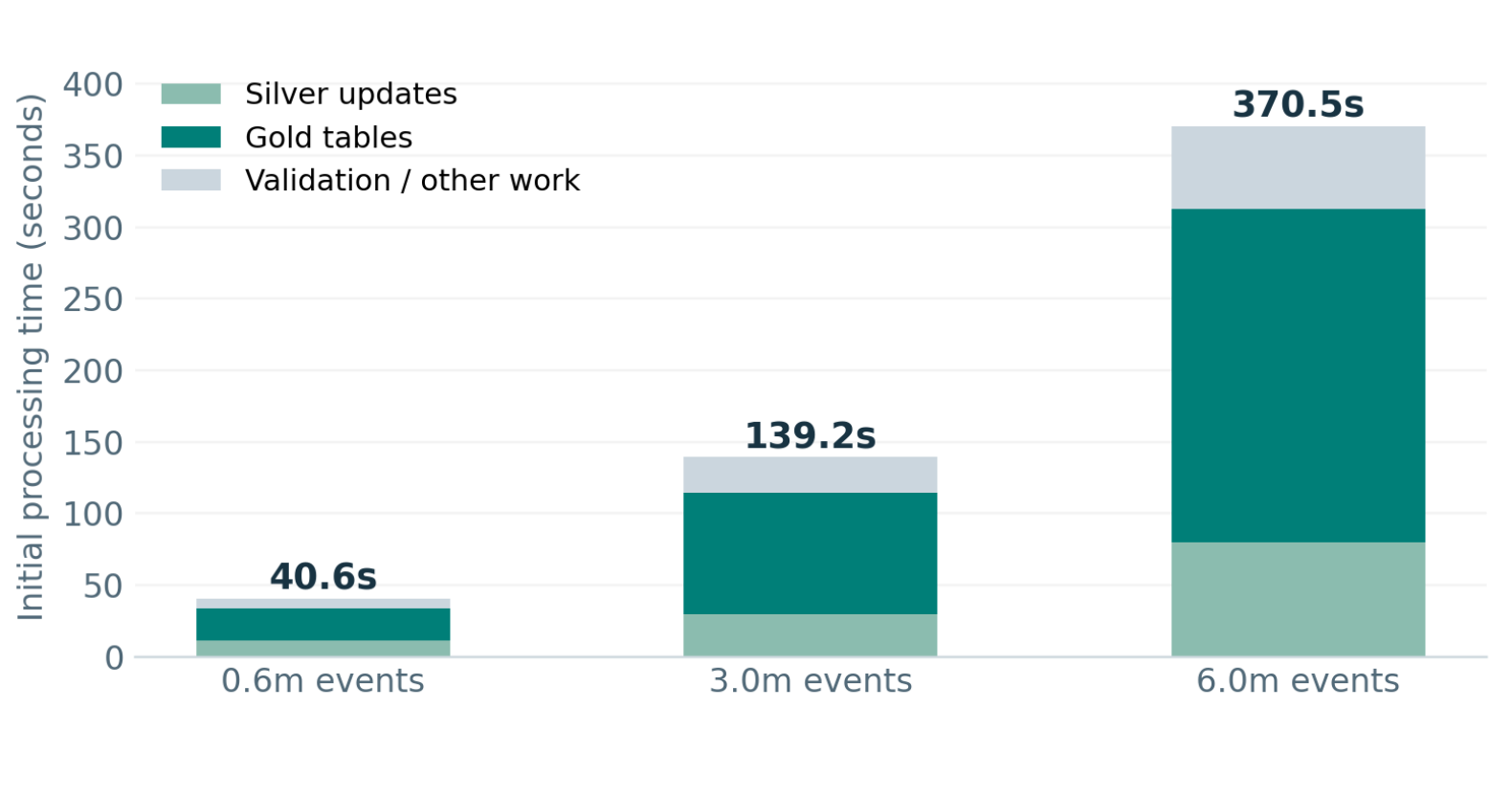
Metric	Simple meaning	What it tells us
Actual input rate	Source events received per second	How much load was really generated
Batch duration	Time to process a group of events	How much work each batch takes
Freshness and backlog	Time until publication; events waiting	Whether processing keeps up
CPU, GC and spill	Compute use, memory cleanup and disk spill	Where resource pressure appears
Failures and counts	Failed tasks and source/Gold agreement	Whether results remain correct

Separate tests: bulk load, short CDC steps, broker recovery and query concurrency.

Sources: docs/performance-stress.md | docs/evidence/stress/streaming-levels.json

PERFORMANCE RESULTS

Large loads: Gold takes most of the time



370.5s

6 million input events

Initial processing time

63%

Time spent in Gold

232.5s of 370.5s

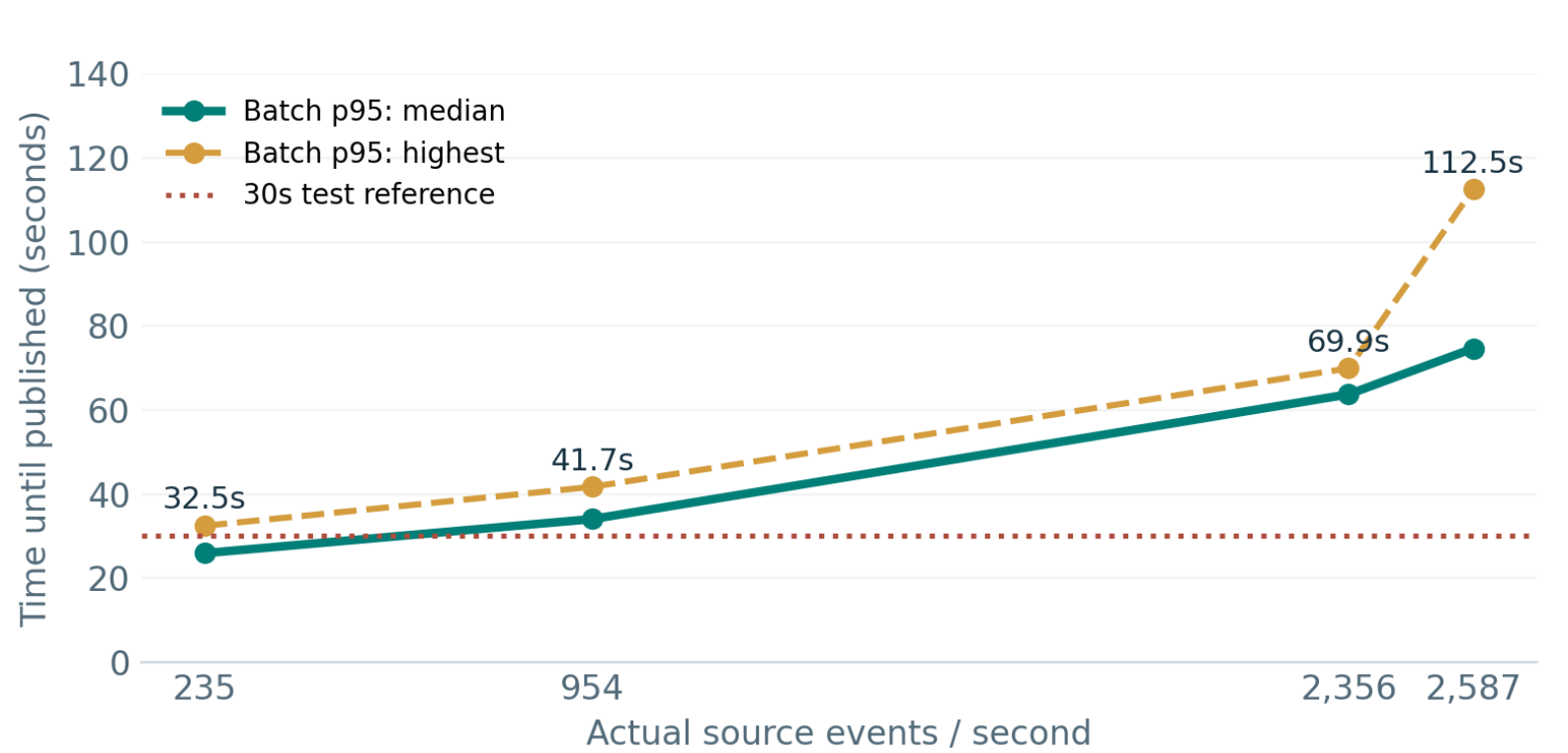
846 MB disk spill
Memory pressure near cap
0 failed task attempts

Largest run: 482.8s task CPU • 51.3s task GC. Next focus: Gold work and target-table scans.

Sources: docs/evidence/stress/materialization.json | docs/evidence/stress/scale-1m-spark-metrics.json

PERFORMANCE RESULTS

Streaming: more input means more delay



Delay grows with load

Above ~2,000 events/s, waiting events increased. Highest batch p95: 112.5s.

The pipeline caught up

All four steps drained after input stopped.
Highest steps: ~66s / ~70s.

p95 covers 95% of a batch's events, not all events globally. Steps ~2 min; 6k/s was not reached.

Sources: docs/evidence/stress/streaming-levels.json | docs/evidence/stress/streaming-batches.json

VALIDATION RESULTS

Correctness and recovery were checked

774,001
Raw events accounted for
Kafka = Bronze = published input

3 / 3
Source comparisons passed
All four business metrics

53.5s
Observed recovery interval
Stop → restart → catch-up → poll

Updates and deletes

12,375 order updates
6,378 item deletes
Source and Gold results matched.

Temporary Kafka outage

6,000 events written while offline
Broker restarted; events recovered
Spark restart was not needed.

Recovery used retained volumes and checkpoints. One broker is not a highly available setup.

Sources: docs/evidence/stress/final-integrity.json | docs/evidence/stress/broker-outage.jsonl

HANDOVER

Delivery summary and next step

Delivered

Streaming pipeline
Delta dimensional tables
CLI + code + tests + evidence

Measured

Large loads completed
Gold is the main cost
Freshness worsens under load

Next

Reduce Gold work
Test real workload and targets
Then size production resources

```
docker compose run --rm --no-deps query metrics all
```

Code, run instructions and evidence

github.com/fabdutra/deel-data-engineering-task

Sources: README.md | docs/presentation/README.md